Process Evidence: Annotated Conversation Trail

What this is. Three segments from my conversations with Claude about this project, October 15 to November 4, 2026. For each turning point, the left column holds a screenshot of the original conversation (my messages were written in Chinese) and the right column holds an English translation. Under each one I note the outcome. The dashed boxes marked "My reasoning" are where I explain, in my own words, why I made each choice.

Where my own decisions show in this record

- Segment 1: I objected that a script could replace the AI in my first idea, then chose to switch the project and defined the new inputs and outputs myself.
- Segment 2: I gave up automatic folder syncing in favor of web uploads, and decided on no accounts and browser storage.
- Segment 3: I noticed the missing camera data myself, ran checks in the browser, and pointed Claude Code at my own photo folder.

Contents

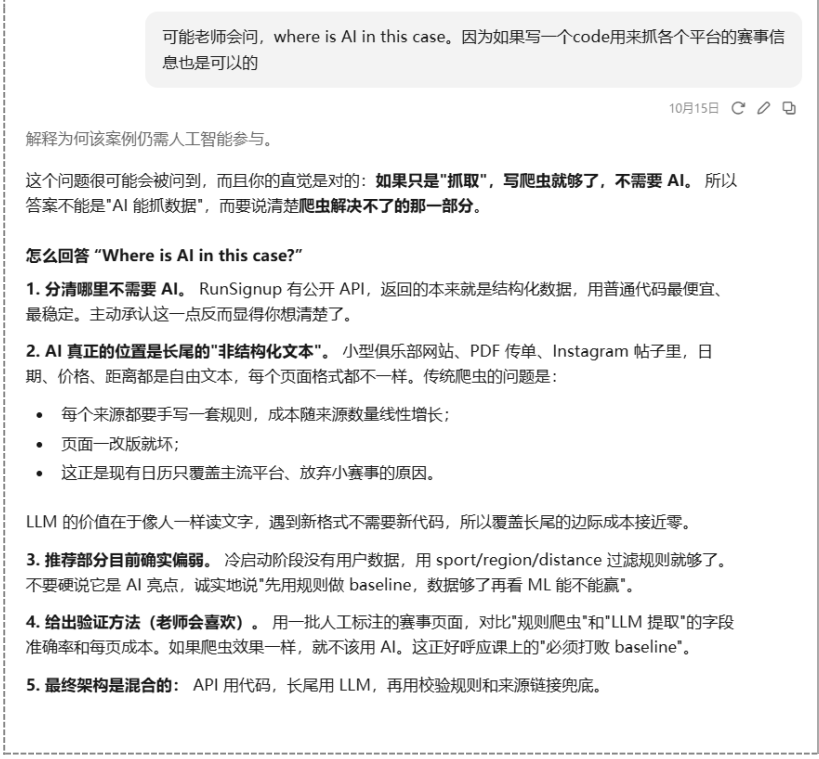
#	Segment	Dates	Tool
1	From StartLine to a photography project	Oct 15	Claude chat
2	Narrowing the scope: from auto-sync to web upload	Oct 15 to 18	Claude chat
3	Failure exhibit: a cut-off response and "no EXIF"	Nov 4	Claude Code

Segment 1: From StartLine to a photography project

October 15 · Claude chat

My first project idea was StartLine, an AI race finder for outdoor athletes. Three exchanges that day moved me away from it.

1.1 The question "where is the AI?"

Original conversation (screenshot)	English translation (by Claude; condensed where marked)
	**Me:** "The teacher may ask 'where is AI in this case', because writing code to scrape race listings from each platform would also work." **Claude:** (condensed) You will probably be asked this, and your instinct is right: if it is only scraping, a crawler is enough and no AI is needed. The answer cannot be "AI can collect the data". It has to name what a crawler cannot do. (1) Say where AI is not needed: RunSignup has a public API that already returns structured data. (2) AI belongs in the long tail of unstructured text: small club sites, PDF flyers, Instagram posts, where every page has a different layout. (3) The recommendation part is weak at the start because a new product has no user data. (4) Test it: compare a rules-based scraper with AI extraction on a labeled sample, and if the scraper matches, drop the AI. (5) The final design is mixed: code for APIs, AI for the long tail, plus validation.

Outcome: *The StartLine write-up now stated where AI was and was not needed, with a test against a scraper baseline. StartLine was dropped later (1.3).*

1.2 The Google Drive exercise

Original conversation (screenshot)	English translation (by Claude; condensed where marked)
	**Me:** "Use Google Drive for this." (after Claude offered a connector) **Claude:** (condensed) I connected to your Google Drive, but I cannot find a folder named "541 Collection" and I found no image files. Likely reasons: the folder was shared with you or is in a shared drive; the name is slightly different; the photos are in Google Photos; or this is a different Google account. Tell me the exact name and where it is and I will continue. **Me:** "It's in Shared with me." **Claude:** (condensed) I sampled 8 of the 135 photos (about 6%) by actually looking at them and reading their camera data, so the conclusions only hold for this sample. I did not check the other photos.

Outcome: *The analysis worked for only 8 photos. Behind the scenes (not visible in the chat), reading an image through the connector returned empty content, so Claude downloaded and decoded the files itself. This became my second failure example.*

1.3 The pivot

Original conversation (screenshot)	English translation (by Claude; condensed where marked)
	**Me:** "I want to switch my final project to the topic we did with Google Drive: analyzing photographs and giving advice. For input, users upload their own photos and photos they like and want to imitate, or point to a folder that is read automatically when updated, with regular reports. Besides the advice I mentioned before, I also want advice on how to shoot to imitate a style I like. Help me think it through and focus on feasibility and the risk of being replaced (how is this different from users uploading photos to an AI agent?)" **Claude:** (condensed) Conclusion: this direction is feasible and easier to demo than StartLine. But as you describe it (users upload photos and the AI analyzes them), it is almost the same as dropping photos into Claude or ChatGPT. Your homework exercise shows this: connecting Claude to a Drive folder took about ten minutes. So the value cannot rest on "AI can analyze photos". It has to rest on what a one-off chat cannot do: collections too large for one conversation (it only sampled 8 of 135), camera data measured by code instead of guessed, memory across visits, analyzing only new photos, and reports. For a single small set the chat is already good, so there is no advantage there.

Outcome: *Project direction changed. I defined three inputs (my photos, reference photos, a folder that updates itself) and added "how to shoot to imitate a style" as an output.*

My reasoning

Why did I move away from StartLine? What did the Drive exercise show me that made me want this topic instead?

Honestly, the connector homework gave me the idea. The assignment said the task did not matter, so I picked the first thing that came to mind: have Claude go through photos I took with my own camera, describe my style, and give me shooting and lens advice. Doing that made me realize this is what I actually want to work on. My first reaction when I chose a task was photography, not race listings.

I also felt the role of AI was much clearer here. With StartLine, I had already asked myself where the AI really is, because a script could collect race data. With photos, the AI has to look at images, describe a style, compare it with another, and turn the difference into advice, which a script cannot do.

The exercise also showed me real limits. Claude only looked at 8 of about 135 photos, and nothing carried over to a later conversation. That gave me concrete problems to work on.

Segment 2: Narrowing the scope, from auto-sync to web upload

October 15 to 18 · Claude chat

I wanted the first version to connect to a local folder or Google Drive and update automatically. This segment shows why I gave that up.

2.1 Is automatic syncing realistic?

Original conversation (screenshot)	English translation (by Claude; condensed where marked)
	**Me:** "Any photography hobbyist. By the end of the term I'd like the MVP to connect to a local folder or Google Drive and update automatically. Do you think that's hard?" **Claude:** (condensed) Not especially hard, but it depends on how you implement the connection. The hard part is not the AI. It is "any photographer can connect". Google Drive: broad access uses a restricted permission that needs verification and a security assessment; the lighter "Picker" option may not give automatic access to photos added later, so it needs a small prototype. Google Photos: basically unworkable, since 2025 third-party apps can only access photos the user picks. A more stable alternative is a small program on the user's computer that watches a local folder; Drive for desktop and Dropbox can sync cloud folders to a local folder, so one watcher covers every cloud service. Limits: thse user must install it and keep it running, and a web app cannot watch a local folder in the background. If you insist on "anyone clicks once to connect Drive and updates automatically", I think that is not realistic within the semester.

Outcome: *Automatic syncing for any user was ruled out for this semester.*

2.2 Web upload, no account

Original conversation (screenshot)	English translation (by Claude; condensed where marked)
	**Me:** "Then make it a web page. Support photo uploads, any number at once, the only difference being analysis speed. Each time there are new photos, upload them and analyze iteratively, and keep the earlier memory." **Claude:** (condensed) This version is easier and more likely to finish on time. Without automatic folder watching, the hardest problems (authorization, background running) disappear. The cost is that "automatic update" is no longer a difference, so the value has to come from large collections, camera data measured by code, memory, and comparison with reference photos. Key design points: shrink images in the browser before sending, read camera data before resizing, store each photo's analysis as memory, and skip duplicates by hash. One decision is yours: should memory live in an account on a server (works across devices, easier to evaluate, more work) or in the user's own browser (simple, strong privacy story, lost if cleared)? **Me:** "No account. Store it in the user's own browser, but I can say account login is a mid-term goal. Write the English proposal using the course structure without deep technical detail, and a developer document I'll give to Claude Code later."

Outcome: *Decisions: web upload; any batch size; only new photos analyzed; no account; storage in the browser; accounts as a mid-term goal. Proposal v1 and developer specification v1 were written (Oct 18).*

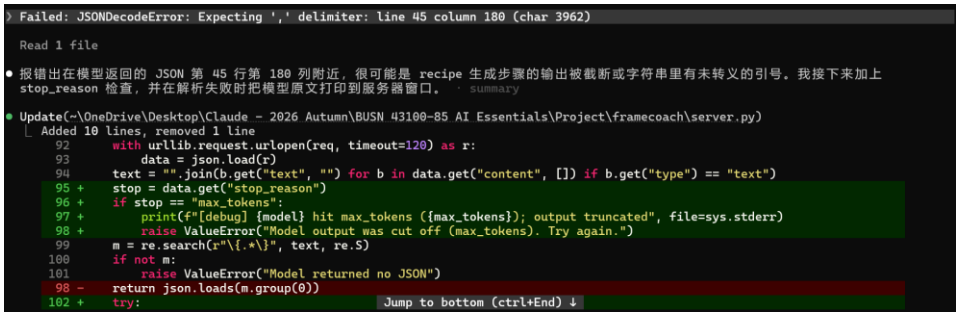
Segment 3: Failure exhibit, a cut-off response and "no EXIF"

November 4 · Claude Code

First real runs of the app. I noticed the missing camera data first, but reported it to Claude Code after the JSON error, so the screenshots appear in that order.

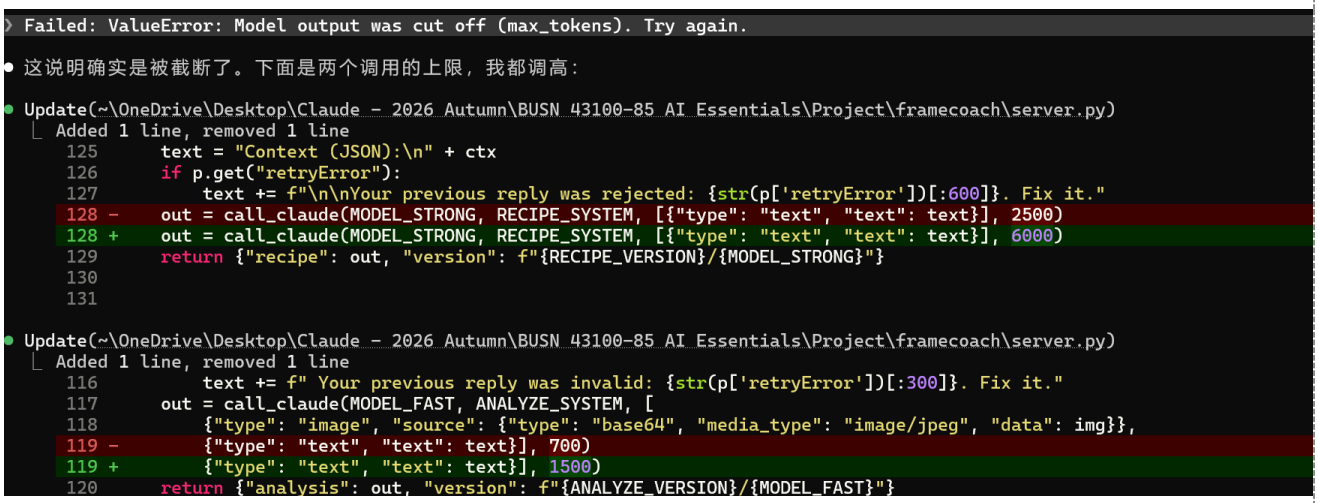
3.1 A JSON error that turned out to be a cut-off response

Original conversation (screenshot) & English translation (by Claude; condensed where marked)



Me: (pasted) "Failed: JSONDecodeError: Expecting ',' delimiter: line 45 column 180 (char 3962)"

Claude Code: (condensed) The error is in the model's JSON around line 45, most likely in the recipe step: the output was cut off or a string contains an unescaped quote. I will add a check for why the model stopped and print the model's raw text when parsing fails. Restart the server, click Generate recipe again, and send me the debug lines. If it says it hit max_tokens, raise that limit; if parsing failed, look for an unescaped quote. This kind of error can be random, so you can record it as a failure example.



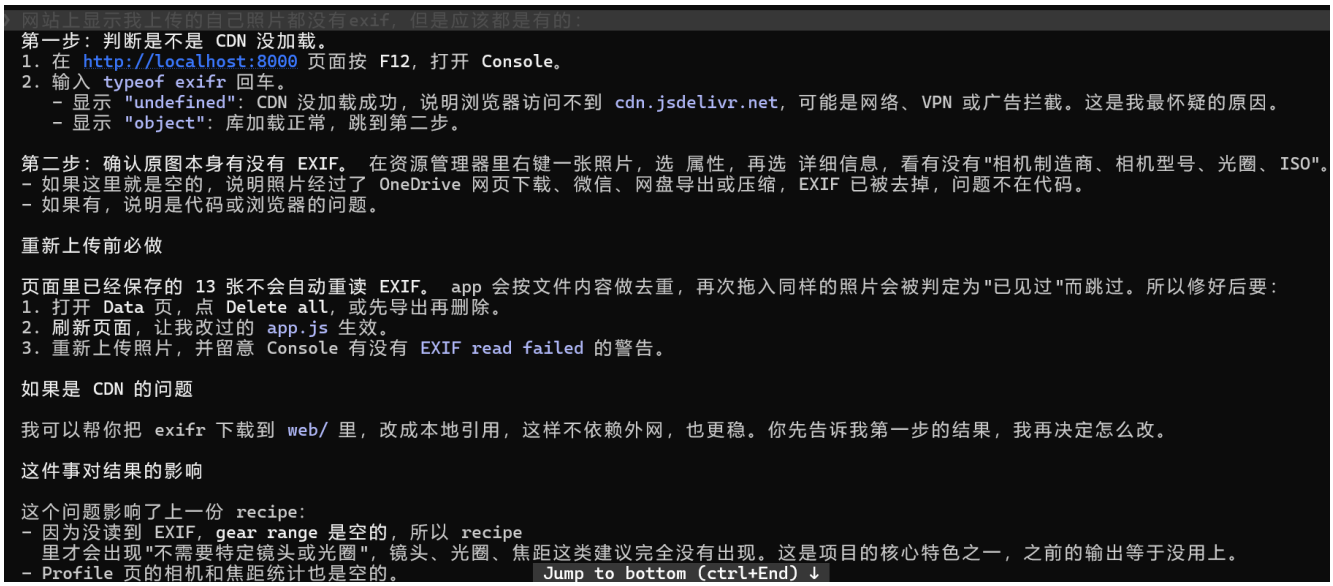
Me: (pasted) "Failed: ValueError: Model output was cut off (max_tokens). Try again."

Claude Code: (condensed) So it really was cut off. I raised the limits: single-photo analysis from 700 to 1500 tokens, recipe generation from 2500 to 6000. A recipe of up to seven items, each with a basis, photo IDs and a label, plus a summary and a weakness, easily exceeds 2500. Raising the limit does not make every answer longer. If it is still cut off, send me the debug output.

Failure: *The response was cut off at the length limit, not malformed by the model. After the limits were raised, recipes were generated. Lesson: diagnose before changing parameters.*

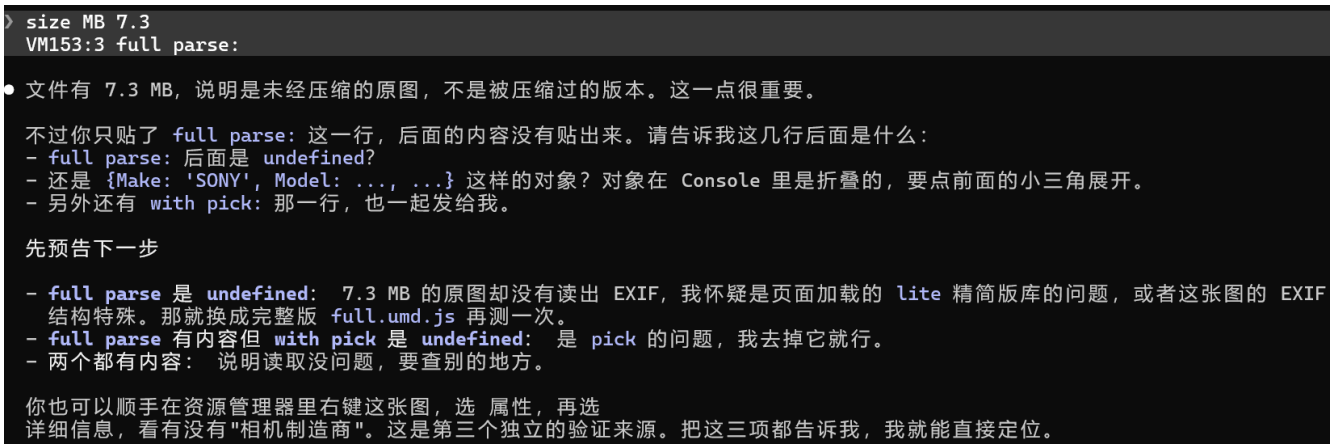
3.2 Every photo showed "no EXIF"

Original conversation (screenshot) & English translation (by Claude; condensed where marked)



Me: "The site shows my own uploaded photos all have no EXIF, but they should all have it." (I had noticed this before trying to generate any recipe and reported it afterward.)

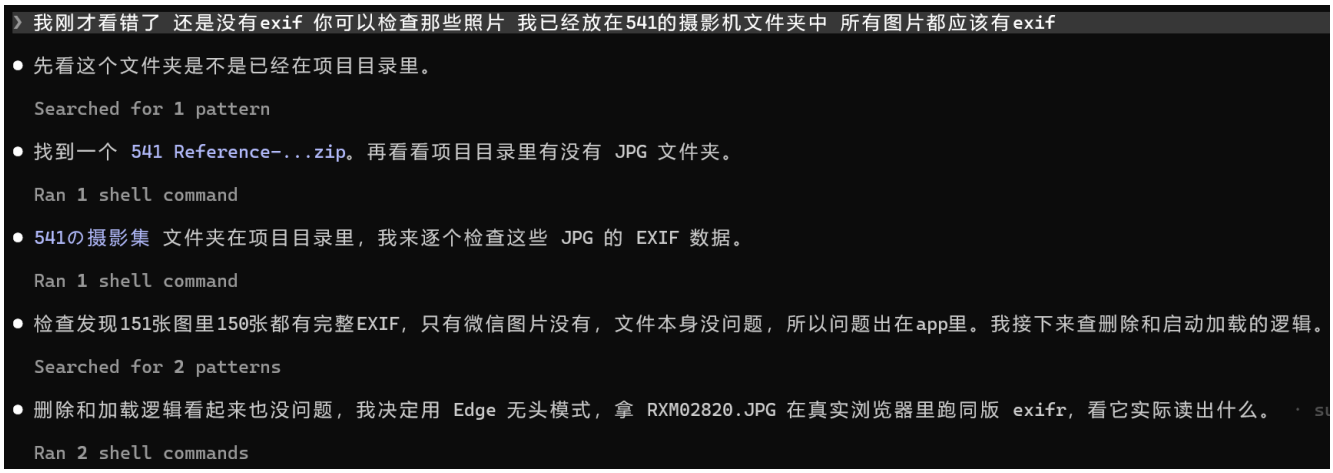
Claude Code: (condensed) Diagnosis. Step 1: is the library from the CDN loaded? Open the page, press F12, type typeof exifr. "undefined" means it did not load; "object" means it loaded. Step 2: check the original file's properties for camera maker, model, aperture and ISO; if they are empty, the file lost its camera data and the code is not the problem. Effect: because no camera data was read, the gear range was empty, so the earlier recipe had no lens, aperture or focal-length advice.



Me: The console shows "object". The photo is 7.3 MB and the full parse returned content.

Claude Code: (condensed) The library loads and reads this file, so the problem is somewhere in the app. I found that app.js hides EXIF errors with an empty try/catch and I am changing it to print a warning.

Original conversation (screenshot) & English translation (by Claude; condensed where marked)



Me: "You can check those photos. I've put them in the 541 camera folder. All the pictures should have EXIF."

Claude Code: (condensed) Found and fixed. The cause is in our code, not your photos. I scanned 151 JPGs: 150 have full camera data and only one WeChat image does not. The bug: passing a pick option to exifr.parse throws a TypeError in this library version, and the empty catch hid it, so the page showed "no EXIF". I reproduced it in a headless browser: with pick, three test photos threw errors; without pick they returned 32, 49 and 48 fields. I changed app.js to read everything and choose the fields ourselves. Lesson: do not swallow errors, and tell "no data" apart from "failed to read".

Failure: *Camera data now appears. The failure looked like a normal result, so I could not tell "no data" from "could not read". It also explained why the first recipe had no gear advice.*